

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

ORGANISATION OF ISLAMIC COOPERATION (OIC)

Department of Computer Science and Engineering (CSE)**MID SEMESTER EXAMINATION****DURATION: 1.5 HOURS****WINTER SEMESTER, 2024-2025****FULL MARKS: 75****CSE 4501: Operating Systems****Programmable calculators are not allowed. Do not write anything on the question paper.**Answer **all 3 (three)** questions. Figures in the right margin indicate full marks of questions with corresponding COs and POs in parentheses.

1. a) Mention the three primary process states with a single-line description of each state. Also, illustrate the process state transition diagram, clearly indicating the transition conditions between the states. 2 + 3
(CO1)
(PO1)
- b) Consider the program shown in Code Snippet 1 where two processes (P_0 and P_1) using a shared variable turn to control access to a critical section. 4 + 5
(CO3)
(PO2)

```

1  int turn; /* Shared variable, initially 0 or 1 */
2
3  /* Process 0 */
4  while (TRUE) {
5      while (turn != 0) /* busy waiting */;
6      critical_region(); /* critical section */
7      turn = 1; /* give turn to process 1 */
8      noncritical_region(); /* remainder section */
9  }
10
11 /* Process 1 */
12 while (TRUE) {
13     while (turn != 1) /* busy waiting */;
14     critical_region(); /* critical section */
15     turn = 0; /* give turn to process 0 */
16     noncritical_region(); /* remainder section */
17 }

```

Code Snippet 1: Pseudocode for Question 1.b

- i. Describe how this algorithm ensures mutual exclusion between P_0 and P_1 .
 - ii. Provide an example scenario where it can violate the condition 3 for achieving mutual exclusion. (Condition 3: No process running outside its critical region may block another process.)
 - iii. Write down Peterson's solution for achieving mutual exclusion and explain how it solves the problem mentioned in question 1(b).ii
- c) The ProducerConsumer problem is a classic example of synchronization using shared resources (a bounded buffer) where, producer generates data and places it into the buffer, consumer removes data from the buffer. Both must operate without interfering with each other or causing overflow/underflow. 4 + 3
(CO3)
(PO2)
- i. Write the semaphore-based pseudocode for both Producer and Consumer processes using three semaphore: mutex (for mutual exclusion), empty (counting semaphore for available slots), full (counting semaphore for filled slots).
 - ii. Describe what happens if the Producer runs much faster than the Consumer, or, the Consumer runs much faster than the Producer.

2. a) Define Deadlock. Mention the four conditions to be present for a deadlock to happen.

b) Consider a system with multiple processes and multiple instances of each resource type.

- Write the deadlock detection algorithm that can identify deadlocks in such a system.
- Consider a system with 5 processes (P_1-P_5) and 3 resource types (R_1, R_2, R_3). Each resource type has multiple instances available. Here, Resource in existence, $E = (3, 2, 3)$. The current resource allocation and request matrices are shown below -

Table 1: Allocation and Request Matrices

Process	R1	R2	R3
P_1	1	0	1
P_2	0	1	0
P_3	1	1	0
P_4	1	0	1
P_5	0	0	1

a Allocation Matrix

Process	R1	R2	R3
P_1	1	0	1
P_2	1	1	0
P_3	0	1	1
P_4	1	0	0
P_5	0	0	1

b Request Matrix

Apply the deadlock detection algorithm from question 2(b).i for the information provided above. Show step-by-step how the algorithm detects any deadlocks.

c) Consider a system with 5 processes (P_1-P_5) and 3 resource types (A, B, C). Current allocation and maximum resource need is shown below -

Table 2: Matrix for Question 2.c

Process	Allocation			Max		
	A	B	C	A	B	C
P_0	3	2	2	5	6	2
P_1	0	1	0	2	1	0
P_2	1	1	1	5	6	1
P_3	3	2	3	9	9	6
P_4	2	1	1	3	2	1

Available Resources, (A, B, C): (2,0,0)

Is the above a safe state or an unsafe? If it is safe, write the safe sequence of process to run. If it is unsafe, mention which processes are able to finish (if any) and what resources are available when these processes complete?

d) Explain how to attack the following conditions to structurally prevent deadlock -

- Hold and Wait Condition
- Circular Wait Condition

3. a) Consider the following workload shown in Table 3.

7.5 × 2

Table 3: A table of items for Question 3.a

(CO2)
(PO3)

Process	Priority	Arrival Time (AT)	Burst Time (BT)
P1	2	0	5
P2	1	1	3
P3	4	2	8
P4	3	3	6
P5	2	4	2

Construct the Gantt chart and calculate the average turnaround time and waiting time for each of the following scheduling algorithm

- Shortest Remaining Time Next
 - Round robin with quantum 3
- b) Illustrate the position of scheduler, process table and thread table in user-level thread implementation and kernel-level thread implementation with diagrams. With the help of your diagrams, justify which thread implementation is better with respect to blocking.

6 + 4
(CO1)
(PO1)